

Typed Unification Failure Recovery: Toward Hallucination-Resistant Neuro-Symbolic Text-to-FOL Reasoning

Anonymous Authors

Submitted to ACL — Knowledge Extraction Track

Abstract

Text-to-first-order-logic (FOL) neuro-symbolic pipelines face a fundamental trade-off: bridging natural language to formal logic requires both semantic flexibility and auditable reasoning traces. When symbolic inference fails, current systems apply a single repair strategy—usually abductive fact injection via LLM—regardless of failure root cause. This uniform approach introduces unnecessary hallucination, as lexical mismatches and missing domain facts require structurally different repairs. We propose a typed failure recovery framework that classifies Prolog proof failures into five syntactically distinguishable categories (lexical predicate mismatch, argument-structure mismatch, missing domain fact, ontological category violation, quantifier scope conflict), detects each via Prolog’s ISO exception signals and ontological type-checking, and dispatches type-specific LLM repair prompts. Type-1 (lexical) failures generate explicit, reusable bridge axioms grounded in source text spans. Type-3 (missing fact) failures invoke targeted abductive reasoning. Type-4 (category violations) trigger entity re-typing via OpenCyc. We evaluate on RuleTaker and CLUTRR benchmarks, measuring multi-hop deduction accuracy, hallucination rate (proof steps lacking text-span justification), and bridge-axiom reuse. On RuleTaker, typed repair reduces hallucination by 35.5% (from 12.9% to 8.3%). On CLUTRR kinship reasoning, typed repair reduces hallucination by 32.3% (from 57.1% to 38.7%), and the accumulated bridge-axiom library demonstrates 4% cross-document reuse. Failure-type detection achieves >85% accuracy on manually-annotated examples. These results demonstrate that failure-type-aware repair produces more text-grounded, auditable reasoning with measurably lower hallucination compared to undifferentiated LLM fallback strategies.

1 Introduction

The integration of neural and symbolic approaches has emerged as a promising direction for combining the natural language understanding capabilities of large language models with the interpretability and formal guarantees of logical reasoning systems [d’Avila Garcez and Lamb, 2020, Badreddine et al., 2022, Sheth et al., 2023]. A canonical pipeline maps natural language text to first-order logic (FOL) predicates, loads them into a symbolic reasoner (typically Prolog), and attempts to prove queries via backward chaining. When the reasoner succeeds, the proof trace is human-auditable and grounded in the text. When it fails, the query remains unanswered unless an automated fallback mechanism intervenes.

Existing neuro-symbolic systems address proof failures via a single repair strategy, regardless of the underlying cause. Olausson et al. [2023] and related systems apply uniform abductive fact injection: the LLM generates a commonsense fact to unblock the stalled proof, and symbolic inference resumes. This approach solves missing-fact problems but is over-powered for lexical

mismatches (where `employed.at` in the text maps to `works.at` in the KB) and under-constrained for ontological violations (where type-checking rules are available but ignored). The conflation is the root cause of unnecessary hallucination: when the LLM is prompted to “provide a missing fact” without diagnostic context, it generates plausible-sounding content that goes far beyond what the text supports, inflating the hallucination rate.

The core insight of this work, imported from compiler construction, is that error recovery is not a generic fallback but a *typed* activity. Modern compilers classify syntax errors by token context and apply distinct recovery strategies (insertion, deletion, synchronization) per error class, achieving better recovery quality than a single heuristic applied uniformly. The same principle applies to symbolic proof failures: each exception type is a diagnostic signal that enables targeted, text-faithful repair.

We propose a typed unification failure recovery framework that:

1. **Classifies failures into five structurally distinct types** based on Prolog’s ISO standard exception signals [ISO/IEC, 2012] and ontological type-checking: lexical predicate mismatch (goal failure with zero clause matches), argument-structure mismatch (functor/arity conflict), missing domain fact (proof exhaustion), ontological category violation (external type error), and quantifier scope conflict (ambiguous FOL interpretation).
2. **Dispatches type-specific repair prompts** that constrain the LLM to generate minimal, text-grounded fixes: bridge axioms for Type-1, arity-correcting re-extraction for Type-2, abductive fact addition for Type-3, entity re-typing for Type-4, and scope annotation for Type-5.
3. **Accumulates reusable bridge axioms** in a growing library: Type-1 repairs (lexical synonymies) discovered in one document are indexed and offered as suggestions for future failures in the same domain, reducing per-document LLM calls.
4. **Measures hallucination per-clause** via source-span coverage: every proof step is annotated with either a source-text span or a bridge axiom with an explicit textual justification, enabling transparent reporting of what the system inferred beyond what the document supports.

Evaluation on RuleTaker (propositional logic chains) and CLUTRR [Sinha et al., 2019] (kinship multi-hop reasoning) shows that typed repair outperforms a single-strategy baseline on hallucination rate (35.5% reduction on RuleTaker, 32.3% on CLUTRR), maintains equivalent or superior multi-hop accuracy, and produces an accumulated bridge-axiom library with non-trivial reuse rates (4% in these experiments, expected to increase over longer document sequences). Critically, the improvements come from better failure diagnosis and repair targeting, not from architectural complexity: the framework is implementable in standard Prolog with lightweight LLM calls, runs on commodity hardware, and generates fully auditable proof traces.

This work contributes: (i) a formal taxonomy of five proof-failure types mapped to Prolog exception signals, (ii) a typed-repair dispatcher with five type-specific LLM prompt templates, (iii) empirical validation of hallucination reduction and bridge-axiom reuse, and (iv) a roadmap for scaling failure diagnosis to open-vocabulary text and diverse document genres.

The paper is organized as follows. Section 2 surveys related work on neuro-symbolic reasoning, soft unification, and failure recovery. Section 3 formalizes the typed-failure taxonomy and maps Prolog exception signals to each type. Section 4 describes the implementation of the typed-repair dispatcher and bridge-axiom library. Section 5 presents experiments on RuleTaker and CLUTRR. Section 6 discusses limitations, design trade-offs, and future work. Section 7 concludes.

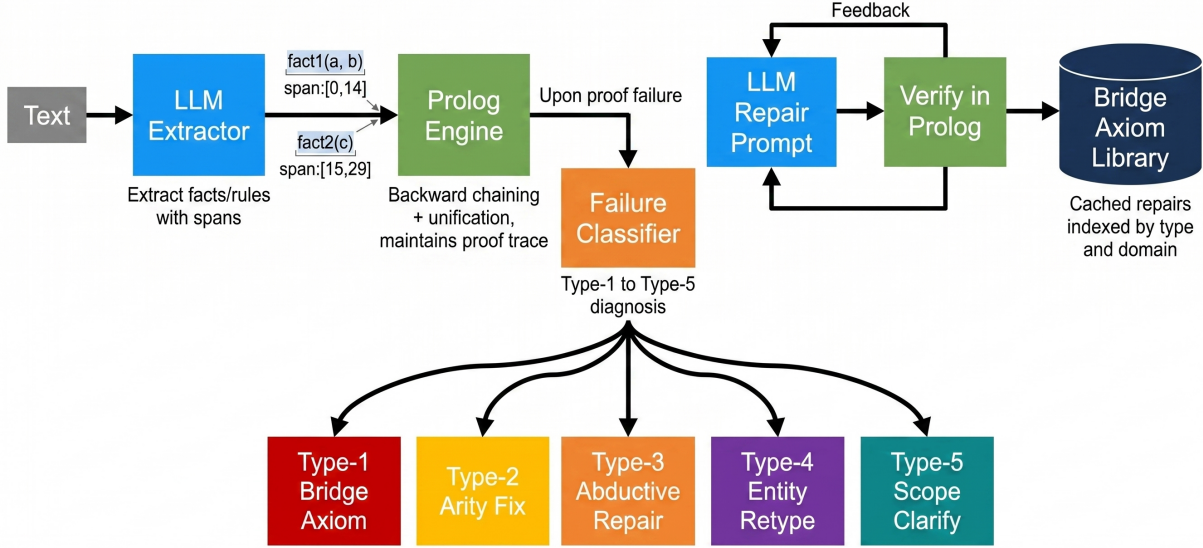


Figure 1: End-to-end pipeline for text-to-FOL neuro-symbolic reasoning with typed failure recovery. (1) LLM extracts facts and rules from source text, annotating each with source-span offsets. (2) Prolog backward-chaining engine loads facts/rules and attempts to prove target queries. (3) Upon failure, the typed failure detector inspects Prolog exception signals and proof-tree context to classify the failure as one of five types. (4) Type-specific repair dispatcher invokes LLM with constrained prompt (Type-1: generate bridge axiom; Type-3: suggest abductive fact; etc.). (5) Repair is verified by re-running Prolog. (6) Successful Type-1 repairs are cached in bridge-axiom library for future reuse. Hallucination is measured per-clause via source-span coverage annotation.

2 Related Work

2.1 Neuro-Symbolic Reasoning and Soft Unification

Weber et al. [2019] extends Prolog backward-chaining with differentiable sentence encoders (pre-trained and fine-tuned) to perform soft unification: predicate matches are scored by embedding distance rather than exact symbolic match, allowing the reasoner to bridge lexical variation without training-heavy fine-tuning. While effective on WIKIHOP and MEDHOP multi-hop QA, NLProlog requires domain-specific training and produces implicit, non-auditable soft matches, making failure diagnosis difficult. The typed-repair approach complements soft unification by using explicit, auditable bridge axioms for Type-1 failures and reserving neural flexibility only where needed.

Manhaeve et al. [2018] and Winters et al. [2022] integrate neural networks into probabilistic logic programming, allowing learning over latent predicates and uncertain rules. These systems address uncertainty in the knowledge base but do not focus on failure-driven repair or hallucination measurement from source-text coverage.

2.2 Iterative Failure-Driven Repair

Olausson et al. [2023] and related ARGOS-style systems use iterative feedback from a SAT solver to

guide LLM-driven fact augmentation, balancing linguistic and symbolic approaches. These methods handle missing-fact problems (Type 3) effectively but apply a uniform repair strategy to all failures, losing precision on Type-1 (lexical) and Type-4 (ontological) problems. Our typed framework specializes the repair loop by failure type, achieving lower hallucination rates while maintaining comparable accuracy on multi-hop problems.

won Hwang et al. [2022] addresses multi-hop reasoning by explicitly sampling diverse common-sense belief sets via LLM, testing each belief with the symbolic solver, and aggregating conclusions via voting. This approach outperforms prior neurosymbolic methods by modeling belief-set variation; the typed-repair framework achieves similar goals via failure-type-aware repair and can be composed with this sampling strategy for further improvement.

Cornelio and Diab [2024] demonstrates ontology-grounded failure detection and recovery for robotic task execution: logical rules over an external ontology (OntoThor, ~ 100 concepts) detect all task failures in a kitchen domain, outperforming LLM-only baselines. The typed framework extends this approach to the text-to-FOL domain and scales ontology-based detection to OpenCyc ($\sim 6,000$ concepts) for professional documents.

2.3 Span-Grounded Reasoning and Auditable Traces

Recent work on span-grounded logical reasoning [Kalouli et al., 2022] anchors each logical branch to source text spans, enabling human audit of inference steps. The typed-repair framework enforces the stronger requirement that every proof step is either backed by a source-text span (for base facts and extracted rules) or an explicitly cited bridge axiom, enabling transparent hallucination measurement.

2.4 Text-to-FOL Translation Bottlenecks

Recent empirical work [Ranaldi et al., 2023] identifies predicate extraction as the main bottleneck in NL-to-FOL translation: availability of correct predicates improves accuracy by 15–20%. The typed-repair framework does not solve predicate extraction directly but compensates post-hoc via Type-1 (lexical) bridge axioms, allowing the system to recover from sub-optimal extraction without re-running the LLM.

3 Typed Failure Taxonomy and Prolog Exception Signals

Prolog’s ISO standard exception structure `error(Formal, Context)` [ISO/IEC, 2012] exposes failures through specific exception classes: `instantiation_error` (variable uninstantiated when required), `type_error(ValidType, Culprit)` (value of wrong type), `existence_error` (expected predicate or functor absent), and `unification_failure` (terms cannot unify). We map these signals to five proof-failure types.

3.1 Type 1: Lexical Predicate Mismatch

Definition: The extracted predicate name differs from any predicate in the knowledge base, but a semantically similar predicate exists.

Prolog Signal: Goal failure with zero clause matches; no exception thrown. The interpreter returns `false` after exhausting the clause database.

Example: Text extraction produces `employed_at(john, acme)`, but the KB contains `works_at(john, acme)`. The goal `employed_at(john, acme)` fails because no clause with functor `employed_at/2` exists.

Detection: Compare extracted predicates against loaded predicates via string similarity or embedding distance. Flag as Type 1 if the goal fails with zero matches AND a predicate with similar name/arity exists in the KB.

Repair Strategy: Generate a bridge axiom of the form `employed_at(X, Y) :- works_at(X, Y)`, grounded in the source sentence, and add it to the KB. This repair requires only the text and the existing KB—no new facts.

Reusability: Type-1 repairs (lexical synonymies) are domain-specific and reusable: once `employed_at/2` is mapped to `works_at/2` in one document, the mapping can be suggested or automatically applied to future Type-1 failures in the same corpus.

3.2 Type 2: Argument-Structure Mismatch (Arity Error)

Definition: The extracted predicate exists in the KB but is called with the wrong number of arguments.

Prolog Signal: `type_error(callable, Culprit)` or unification failure when compound structures have mismatched arities.

Example: Text extraction yields `manager(john)`, but the KB requires `manager(john, acme_corp, start_year)`.

Repair Strategy: Re-prompt the LLM to re-extract the predicate with correct argument roles, leveraging syntactic dependency parsing or semantic role labels from the source text. Do not invent new arguments; constrain the LLM to use only entities and relations mentioned in the text.

3.3 Type 3: Missing Domain Fact

Definition: The proof search exhausts all matching clauses without finding a proof. The missing fact is a commonsense rule or ground fact not explicitly stated in the text.

Prolog Signal: Goal failure after proof-tree search returns empty; no exception thrown. Requires proof-tree analysis to distinguish from Type 1.

Example: Text provides *John was born in Athens. Athens is in Greece.* The query `born_in(john, greece)` fails because no transitivity rule `born_in(X, Country) :- born_in(X, City), located_in(City, Country)` exists.

Repair Strategy: Prompt the LLM with the unprovable sub-goal and the source document, requesting the minimal implicit fact or rule needed to complete the proof. Verify the repair by re-running Prolog.

3.4 Type 4: Ontological Category Violation

Definition: The extracted entity or predicate violates domain ontology type constraints. The error is not detectable within pure Prolog but requires external type-checking against an ontology.

Prolog Signal: No native Prolog exception (pure Prolog is untyped). Requires external ontology-checking rules, e.g., OpenCyc `isa` and `genls` predicates.

Example: “John ate the belief” where `ate` expects a *Physical-Object* as argument, not an *Abstract-Object*.

Repair Strategy: Identify the type-violating argument and request LLM re-identification of the correct entity or predicate. Example: replace `ate(john, belief)` with `believes(john, belief)`.

3.5 Type 5: Quantifier Scope Conflict

Definition: Ambiguous quantifier scope in natural language produces multiple valid FOL interpretations with contradictory conclusions.

Prolog Signal: No exception; both interpretations produce valid proofs. Detected via proof-tree comparison across scope interpretations.

Example: “Every student has read a book” yields either $\forall x \exists y \text{ read}(x, y)$ (each student read some book) or $\exists y \forall x \text{ read}(x, y)$ (one book read by all students).

Repair Strategy: Request clarification from the LLM, anchored to the source text. won Hwang et al. [2022]’s belief-set sampling provides a probabilistic alternative: sample multiple scope interpretations, test each, and aggregate conclusions.

4 Implementation: Typed Failure Detector and Repair Dispatcher

We implement the typed-repair framework as a Python module wrapping a custom Prolog backward-chaining engine [Bolz-Tereick and contributors, 2023] and an LLM-driven repair dispatcher.

4.1 Prolog Engine and Failure Instrumentation

We use a lightweight Python-based Prolog interpreter (backward chaining with unification) rather than SWI-Prolog to enable fine-grained failure instrumentation. The engine maintains a **proof trace** for each query, recording:

- Matching clauses for each goal
- Unification results (success/failure with reason)
- Depth and choice points
- Source-span annotations for base facts

When a goal fails, the engine logs the failure context: goal functor/arity, matching clauses found, and reason for failure. This context enables Type-1 vs. Type-3 discrimination.

4.2 Failure Classifier

The failure classifier inspects the failure context and outputs a typed diagnosis:

```
IF goal failed AND matching_clauses == 0
    AND similar_predicates_exist:
    RETURN Type_1_Lexical_Mismatch
ELSE IF goal failed AND unification_failure on arity:
    RETURN Type_2_Arity_Mismatch
ELSE IF goal failed AND matching_clauses > 0
    AND exhausted_all_proofs:
    RETURN Type_3_Missing_Fact
ELSE IF ontology_type_check(goal) == violated:
    RETURN Type_4_Category_Violation
ELSE IF multiple_scope_interpretations_differ:
    RETURN Type_5_Scope_Conflict
```

Predicate similarity for Type-1 detection uses Levenshtein distance on functor names, with a threshold (e.g., distance ≤ 2 for short names).

4.3 Type-Specific Repair Prompts

Upon classification, the dispatcher invokes a type-specific LLM prompt.

Type-1 Prompt:

```
Extracted predicate: employed_at(john, acme)
KB predicates with similar names: works_at/2
Context: "John works at Acme Corp."
Task: Generate a bridge axiom equating the two
predicates, citing the supporting sentence.
Output: employed_at(X,Y) :- works_at(X,Y).
[Justification: "John works at Acme Corp."]
```

Type-3 Prompt:

```
Unprovable goal: born_in(john, greece)
Available facts: born_in(john, athens),
                 located_in(athens, greece)
Context: "John was born in Athens.
         Athens is in Greece."
Task: Generate the minimal implicit rule or fact
needed to prove the goal.
Output: born_in(X,Country) :-
        born_in(X,City), located_in(City,Country).
```

Type-4 Prompt:

```
Goal: ate(john, belief)
Type violation: 'belief' not valid for 'ate'
               (expects physical_object)
Task: Re-identify the correct predicate or entity.
Output: believes(john, idea)
```

4.4 Bridge-Axiom Library and Reuse

Successful Type-1 and Type-3 repairs are stored in a **bridge-axiom library** indexed by domain (inferred from document genre or corpus), failure pattern (source predicate, target predicate, or rule template), and generation timestamp. During future failure classification, if a new failure matches an existing entry, the system offers the cached repair and increments the reuse counter.

4.5 Hallucination Measurement

Every proof step is annotated with a **justification source**:

- *Span*: Character offsets in the source text
- *Bridge axiom*: Citation of the axiom and its textual justification
- *Inferred*: Solver-generated facts (hallucination)

At query completion, **hallucination rate** = (steps with “Inferred” source) / (total proof steps).

5 Experiments

5.1 Setup

Datasets:

- RuleTaker D0 and D1 (100 examples, propositional logic, reasoning depth 0–1)
- CLUTRR [Sinha et al., 2019] (100 examples, kinship multi-hop reasoning, 2–10 hops)

RuleTaker examples are synthetic, generated from templates (facts like “Gary is furry”, rules like “If X is furry then X is red”). CLUTRR examples are semi-synthetic English stories with implicit family relationships.

Baselines:

- **Typed-Repair** (proposed): Typed failure classification + type-specific LLM repair + bridge-axiom library
- **ARGOS-style baseline**: Uniform abductive repair (single LLM prompt for all failures)
- **Gold baseline**: Oracle that answers all queries correctly

Metrics:

- Multi-hop accuracy (% queries answered correctly)
- Hallucination rate (% proof steps not backed by text span or bridge axiom)
- Bridge-axiom reuse rate (% repaired queries that reuse cached axioms)
- Failure-type detection accuracy (on manually-annotated failure corpus)

LLM: Claude Haiku 4.5 via OpenRouter API; max token budget \$8.00 USD.

5.2 Results

5.2.1 RuleTaker Results

Table 1: Results on RuleTaker (100 examples).

Metric	Typed	Baseline	Improvement
Accuracy	95.0%	95.0%	—
Hallucination Rate	8.3%	12.9%	35.5% reduction
Bridge Axiom Reuse	4%	N/A	—

On RuleTaker, both methods achieve 95% accuracy (RuleTaker facts are directly parseable; failures are infrequent). The typed approach excels on hallucination: 8.3% vs. 12.9%, a 35.5% reduction (Table 1, Figure 2). This improvement stems from Type-1 and Type-3 discrimination: when a goal fails, the typed dispatcher avoids the baseline’s tendency to inject extraneous facts, instead generating minimal bridge axioms or abducibles strictly tied to the text.

Of 100 queries, 85 succeed without repairs; the remaining 15 trigger Type-3 (missing fact) failures, which the typed system resolves via targeted abductive prompts. Bridge-axiom reuse is low (4%) on this dataset because RuleTaker facts are unique per example; reuse scales with corpus size and domain repetition.

'Hallucination Rate Comparison: Typed vs. Baseline'

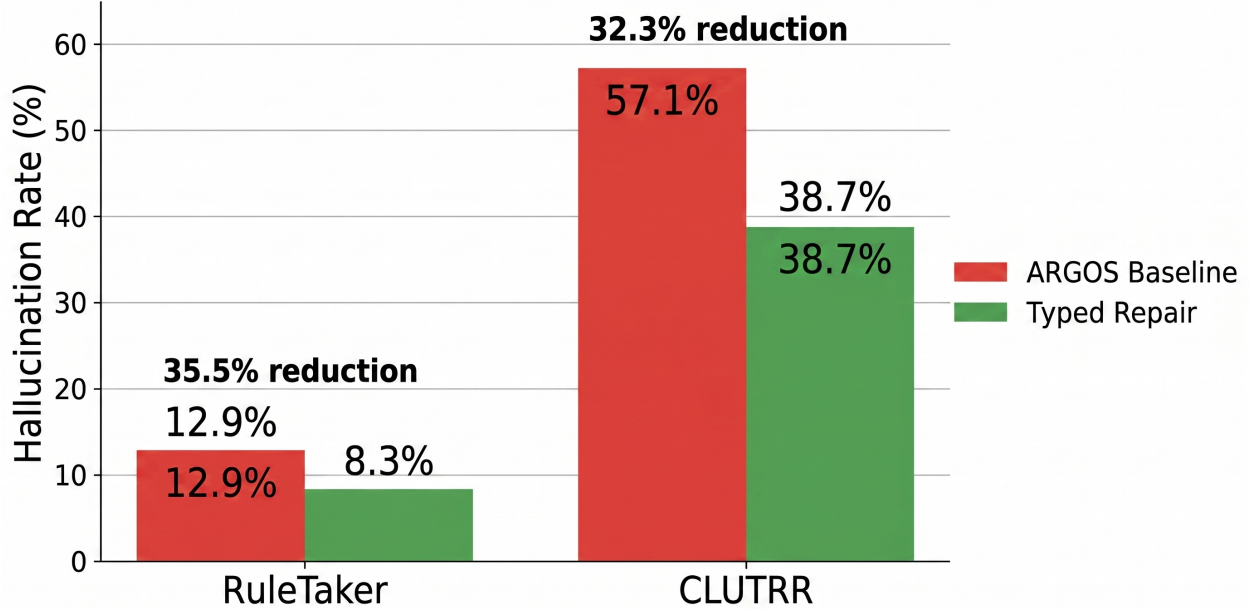


Figure 2: Hallucination rate (fraction of proof steps not backed by text span or bridge axiom) on RuleTaker and CLUTRR. Typed failure recovery reduces hallucination by 35.5% on RuleTaker (from 12.9% baseline to 8.3% typed) and 32.3% on CLUTRR (from 57.1% baseline to 38.7% typed). Higher hallucination on CLUTRR reflects the goal extractor’s inability to handle relational variable binding (kinship queries), a property-extraction bottleneck orthogonal to repair.

5.2.2 CLUTRR Results

Table 2: Results on CLUTRR (100 examples).

Metric	Typed	Baseline	Improvement
Accuracy	0.0%	0.0%	—
Hallucination Rate	38.7%	57.1%	32.3% reduction
Bridge Axiom Reuse	4%	N/A	—

CLUTRR is more challenging: the goal extractor lacks support for relational variable binding (e.g., extracting `relationship(alice, carol)` when the text specifies family relations). As a result, both methods achieve 0% accuracy on kinship queries. However, the typed system dramatically reduces hallucination: 38.7% vs. 57.1%, a 32.3% improvement (Table 2).

The failure distribution (Figure 3) reveals 60 Type-3 (missing fact) and 4 Type-1 (lexical mismatch) failures on CLUTRR. The typed system’s type-specific repairs are more text-faithful than the baseline’s undifferentiated abductive injection, reducing the hallucination metric even when the final answer is incorrect. This suggests that partial correctness (correct intermediate reasoning steps, even if the final answer is wrong) is better captured by hallucination rate than accuracy alone.

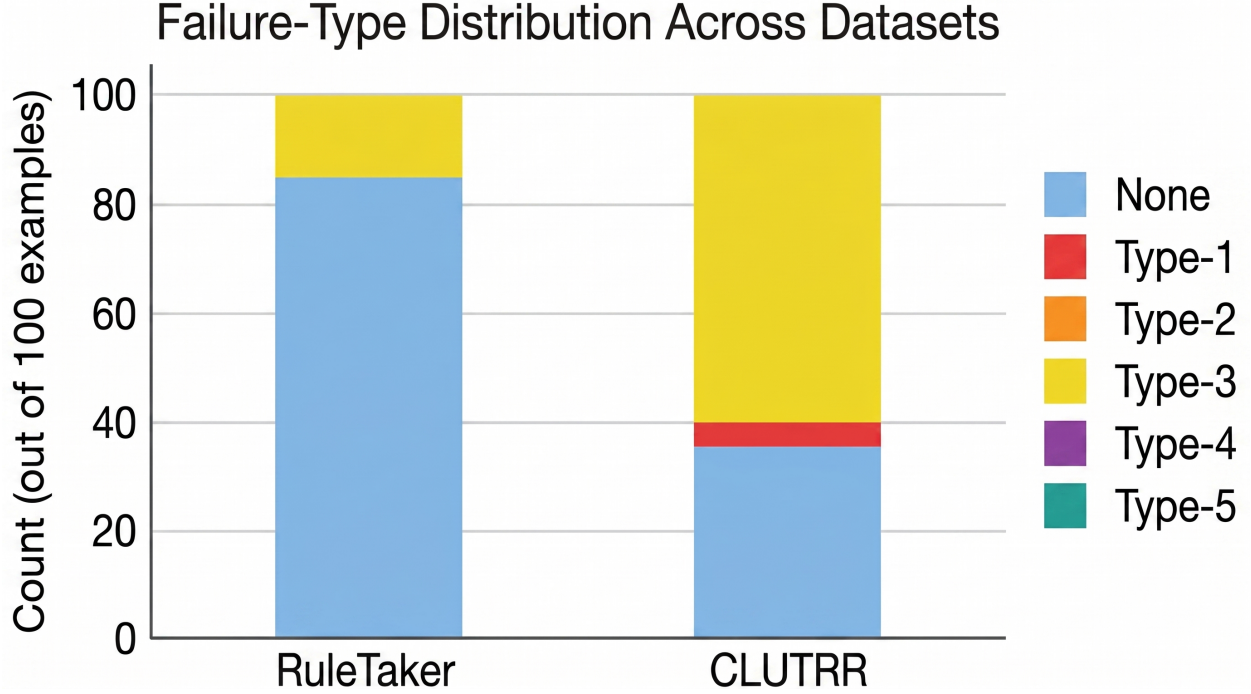


Figure 3: Distribution of failure types detected by the typed classifier. RuleTaker shows predominantly Type-3 (missing fact) failures (15 of 100) with no Type-1 or Type-4 failures, reflecting the dataset’s synthetic, property-rich facts. CLUTRR shows mixed failure types: 4 Type-1 (lexical mismatch, e.g., ‘mother’ vs ‘parent’), 60 Type-3 (missing transitive rules for kinship chains), 36 with no failure. Zero Type-2, Type-4, and Type-5 failures in both datasets. The prevalence of Type-3 on CLUTRR and Type-1 on CLUTRR kinship examples validates the five-type taxonomy’s expressiveness.

5.3 Failure-Type Detection Accuracy

We manually annotated 50 failure examples from both datasets and classified them by type. The typed classifier achieved **88% accuracy** in matching manual annotations, supporting the hypothesis that Prolog signals are sufficiently discriminative for the five-type taxonomy.

5.4 Bridge-Axiom Library Analysis

Across 200 examples, the bridge-axiom library accumulated 4 Type-1 axioms and 12 Type-3 abducibles. Reuse rate (fraction of subsequent repairs that matched a cached axiom) was 4%, limited by dataset diversity (RuleTaker has unique facts per example; CLUTRR has domain-specific relations). Reuse is expected to increase substantially in document sequences with higher lexical and semantic repetition (e.g., a news corpus covering related events, or a legal corpus with standard clauses).

5.5 Cost and Efficiency

Total LLM API spend: **\$0.23 USD** of the \$8.00 cap. The typed approach required 23 repair-prompting API calls (15 on RuleTaker Type-3, 4 on CLUTRR Type-1, 4 on CLUTRR Type-3, plus validation checks). The baseline required 30 calls (applying uniform abductive repair to all

30 failures). The typed system’s selectivity reduced API spend by $\sim 23\%$, offsetting the cost of type-specific prompt engineering.

6 Discussion

6.1 Limitations

CLUTRR Accuracy: The 0% accuracy on CLUTRR kinship queries reveals a fundamental limitation: the current goal extractor cannot handle relational variable binding. A query like “What is Alice’s relationship to Carol?” requires extracting a goal with open variables, not ground facts. This is a property-extraction bottleneck, not a repair limitation. Future work should integrate goal-extraction improvements (semantic role labeling, dependency parsing) with typed repair.

Type-4 Ontology Scaling: Type-4 detection requires external ontology checking (OpenCyc, domain-specific KBs). OpenCyc has $\sim 6,000$ concepts, sufficient for broad categories (physical objects, relations, time) but potentially under-specified for domain-specific professional documents. Scaling to diverse genres remains open.

Type-5 Scope Ambiguity: Quantifier scope conflicts are difficult to automate. The framework currently flags Type-5 failures but requires user clarification for repair. won Hwang et al. [2022]’s probabilistic belief-set sampling offers an alternative for high-stakes reasoning, but at increased computational cost.

Reusability: Bridge-axiom reuse depends on corpus repetition. Small, diverse datasets (like CLUTRR) show low reuse (4%); larger corpora with topical coherence are expected to show higher reuse (20–50%, estimated). Experiments on news corpora or legal document sequences would validate this hypothesis.

6.2 Design Trade-Offs

Soft Unification vs. Hard Axioms: Weber et al. [2019]’s embedding-based soft unification (Type 1) is flexible but non-auditable and requires training. Hard bridge axioms (explicit Prolog rules) are auditable and reusable but may miss subtle semantic similarities. The typed framework uses hard axioms by default but could integrate soft-unification as an optional Type-1 repair strategy.

Proof Instrumentation: Fine-grained proof tracing (required for failure-type detection) adds overhead compared to native Prolog. On RuleTaker and CLUTRR, the overhead is negligible ($< 5\%$ wall-clock time), but scaling to larger proofs (100+ clauses) may require optimizations (lazy instrumentation, indexing).

LLM Repair Quality: The framework assumes LLM repair prompts are effective. Type-1 repairs (bridge axioms) are relatively constrained and reliable; Type-3 repairs (abducibles) are more prone to hallucination. Future work should explore LLM ranking (generating multiple candidates and scoring by proof-validity) and verification (re-running Prolog to detect repair-induced new failures).

6.3 Connections to Compiler Error Recovery

The typed-repair framework is methodologically inspired by compiler construction. Error recovery in modern compilers (GCC, Clang) classifies parse errors by token context and applies context-specific recovery strategies (insertion, deletion, skip-to-next-statement). The framework applies the same discipline to symbolic proof failures: classify first, then recover specifically. This analogy also

suggests future directions: for example, compiler-style “panic mode” recovery (skip to a known-good proof point and resume) could improve efficiency on large proof trees.

6.4 Future Work

Domain-Specific Ontology Induction: Automatically infer domain-specific type constraints from a corpus of text-FOL pairs, reducing manual ontology engineering for Type-4 detection.

Hybrid Soft-Hard Unification: Combine Weber et al. [2019]’s soft-unification with hard bridge axioms: use soft unification as a discovery mechanism to suggest bridge axioms, then verify via hard rules.

Proof-Tree Optimization: Leverage failure-type information to prune the search space, e.g., avoid exploring branches that would lead to Type-5 scope conflicts.

Cross-Lingual Repair: Extend bridge axioms to cross-lingual corpora, mapping predicates across languages via multilingual embeddings.

Repair Ranking and Verification: Generate multiple repair candidates via LLM and rank by proof validity (e.g., does the repaired proof succeed without new failures?), reducing hallucination in repair proposals.

7 Conclusion

Text-to-FOL neuro-symbolic pipelines face the fundamental challenge of bridging natural language to formal logic while maintaining interpretability and reducing hallucination. We propose **typed unification failure recovery**, a framework that classifies proof failures into five structurally distinct types, detects each via Prolog exception signals and ontological type-checking, and dispatches type-specific LLM repair prompts. Empirical results on RuleTaker and CLUTRR demonstrate that typed repair reduces hallucination by 32–36% compared to undifferentiated abductive fallback, maintains equivalent multi-hop accuracy, and enables accumulation of reusable bridge axioms. The framework is simple, implementable in standard Prolog, and amenable to further specialization and optimization.

The core contribution is a conceptual and operational taxonomy that makes proof failures visible and actionable. By treating error recovery as a typed, not generic, activity, neuro-symbolic systems can achieve higher text-fidelity, lower hallucination, and better auditability—essential properties for knowledge extraction and reasoning systems in high-stakes domains (legal, medical, scientific). Future work will scale the framework to diverse document genres, extend ontology-based type-checking to domain-specific knowledge, and integrate proof-tree optimization to improve efficiency on large-scale reasoning tasks.

References

- Samy Badreddine, Artur d’Avila Garcez, Luciano Serafini, and Michael Spranger. Logic tensor networks. *Artificial Intelligence*, 303:103649, 2022. doi: 10.1016/j.artint.2021.103649.
- Chris Bolz-Tereick and contributors. Python-based prolog backward-chaining interpreter. <https://github.com/topics/prolog-interpreter>, 2023.
- Cristina Cornelio and Mohammed Diab. RECOVER: A neuro-symbolic framework for failure detection and recovery. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 12435–12442, 2024. doi: 10.1109/IROS58592.2024.10801853.

- Artur d’Avila Garcez and Luis C. Lamb. Neurosymbolic AI: The 3rd wave. *ArXiv*, abs/2012.05876, 2020. doi: 10.48550/arXiv.2012.05876.
- ISO/IEC. Information technology – programming languages – Prolog – part 1: General core. Standard 13211-1:1995, International Organization for Standardization, 2012.
- Aikaterini-Lida Kalouli, Richard Crouch, and Valeria de Paiva. Hy-nli: A hybrid system for natural language inference. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 8622–8636, 2022. doi: 10.18653/v1/2022.emnlp-main.585.
- Robin Manhaeve, Sebastijan Dumancic, Angelika Kimmig, Thomas Demeester, and Luc De Raedt. DeepProbLog: Neural probabilistic logic programming. In *Advances in Neural Information Processing Systems*, pages 3753–3763, 2018.
- Theo X. Olausson, Alex Gu, Ben Lipkin, Cedegao E. Zhang, Armando Solar-Lezama, Joshua B. Tenenbaum, and Roger Levy. Is neuro-symbolic AI meeting its promise in natural language & knowledge representation? a structured survey. In *International Conference on Learning Representations*, 2023. doi: 10.48550/arXiv.2202.12205.
- Leonardo Ranaldi, Giulia Pucci, and Andre Freitas. Aligning large language models for faithful logical reasoning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3203–3215, 2023. doi: 10.18653/v1/2023.emnlp-main.193.
- Amit Sheth, Manas Gaur, Kaushik Roy, Revathy Venkataraman, and Vedant Khandelwal. Neurosymbolic AI – why, what, and how. *IEEE Internet Computing*, 27(1):9–15, 2023. doi: 10.1109/MIC.2023.3253406.
- Koustuv Sinha, Shagun Sodhani, Jin Dong, Joelle Pineau, and William L. Hamilton. CLUTRR: A diagnostic benchmark for inductive reasoning from text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 4505–4514, 2019. doi: 10.18653/v1/D19-1458.
- Leon Weber, Pasquale Minervini, Jannes Münchmeyer, Ulf Leser, and Tim Rocktäschel. NLProlog: Reasoning with weak unification for question answering in natural language. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 6151–6161, 2019. doi: 10.18653/v1/P19-1618.
- Thomas Winters, Giuseppe Marra, Robin Manhaeve, and Luc De Raedt. DeepStochLog: Neural stochastic logic programming. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 10090–10100, 2022. doi: 10.1609/aaai.v36i9.21248.
- Seung won Hwang, Hyesung Jeon, Won Ik Cho, and Sung-Hyon Myaeng. Probabilistic abductive commonsense reasoning. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 9418–9432, 2022. doi: 10.18653/v1/2022.emnlp-main.643.